CardDiffBench: Evaluating AgentCard Control-Plane Differentials in Agent-to-Agent Systems

Anonymous submission

Abstract

Agent-to-agent systems increasingly rely on AgentCards to expose a remote agent's identity, skills, interfaces, tenants, authorization requirements, and output modes. This control plane enables flexible discovery and routing, but it also creates a failure mode that is not captured by prompt-injection benchmarks: a host agent may act on inconsistent AgentCard state. We introduce CardDiffBench, a benchmark for evaluating whether a host makes unsafe control-plane decisions when public cards, authenticated extended cards, interface lists, tenant bindings, skill-level security, and artifact modes diverge. CardDiffBench defines eight AgentCard differential attacks grouped into extended-card differentials, interface and tenant confusion, and policy/mode drift. The benchmark contains 1,200 scenario-adapted base cases and 3,600 protocol-state perturbations across travel, healthcare, and finance domains. Each case runs in a local A2A-style HTTP environment and is scored with event-level oracles over skill invocation, selected endpoint, tenant binding, protocol binding, scope use, and artifact acceptance. In a completed evaluation of an LLM-backed host using `gpt-5-mini`, attacks succeeded on 3,591 of 3,600 perturbed trials, yielding a 99.75% attack success rate. These results indicate that host-side control-plane validation is a distinct and urgent security problem for interoperable agent ecosystems. CardDiffBench provides a reproducible testbed for measuring that problem and for evaluating defenses that bind decisions to explicit card provenance, tenant identity, authorization scope, and output-mode policy.

1 Introduction

Agent-to-agent (A2A) protocols are becoming a practical substrate for multi-agent systems: one agent can discover another agent, inspect its declared capabilities, select a skill and endpoint, submit a task, and consume returned artifacts. This design shifts part of the trust decision from natural-language prompting to protocol metadata. In particular, an AgentCard may describe a remote agent's public identity, skills, supported interfaces, authentication requirements, tenant bindings, protocol versions, and output modes. A host that routes work through these fields is not only solving a language task; it is making a control-plane decision.

Existing work on agent security has emphasized malicious prompts, untrusted tool output, agent impersonation, and harmful artifacts (TODOCITE: prompt injection and A2A security literature). These risks remain important, but they do not fully cover AgentCard-mediated systems. A host may behave unsafely even when every individual card is schema-valid and every user request appears plausible. The failure arises when two views of the same agent disagree: a public card advertises one set of skills, an authenticated extended card adds another; an interface list changes ordering or tenant labels; a skill requires a stronger scope than the agent-level metadata suggests; or an artifact arrives in a MIME type outside the requested output modes. In such settings, the host must decide which source of truth governs the workflow.

This paper studies *AgentCard control-plane differentials*: security-relevant disagreements among card fields, authenticated views, routing surfaces, tenant bindings, authorization scopes, and artifact modes. We ask a concrete question: when a host is exposed to schema-valid but inconsistent AgentCard state, does it preserve the intended security boundary or complete the workflow through the risky surface?

We present CardDiffBench, a benchmark and harness for this question. The benchmark is inspired by the scenario-adapter style of prior A2A security benchmarking (TODOCITE: A2ASecBench), but it is a separate contribution: it defines its own attack taxonomy, case schema, perturbation layer, event-level scoring oracles, and host evaluation harness. The current frozen dataset contains 1,200 base cases and 3,600 perturbed cases across three high-stakes domains: travel, healthcare, and finance. Each case is generated from a domain scenario and an attack vector, then converted into executable AgentCard state and a natural-language task. Perturbations modify protocol fields, not only wording, while preserving the private oracle.

Our evaluation shows that the problem is not merely hypothetical. We evaluated an LLM-backed host that performs normal A2A discovery, optionally fetches an extended card, chooses a skill and interface, sends a message, and decides whether returned artifacts should be accepted. Across all 3,600 perturbed cases, `gpt-5-mini` succeeded from the attacker's perspective in 3,591 cases, corresponding to a 99.75% attack success rate. All parse failures were zero; the seven unsuccessful trials were operational timeouts or blocked executions. The result suggests that a helpful host

policy that treats authenticated active card state as primary operational evidence is highly vulnerable to control-plane differentials.

The contributions of this work are:

1. We define AgentCard control-plane differentials as a benchmarkable security problem for A2A systems.
2. We introduce an eight-attack taxonomy covering extended-card differentials, interface/tenant confusion, and policy/mode drift.
3. We release a reproducible benchmark construction with 1,200 base cases, 3,600 oracle-preserving protocol perturbations, and event-level scorers.
4. We report a full LLM-backed host evaluation showing 99.75% attack success on `gpt-5-mini`, motivating explicit control-plane defenses.

Positioning against prior work. Security benchmarks for language-model agents often evaluate whether an agent follows malicious instructions, leaks secrets, uses a tool unsafely, or trusts untrusted tool output (TODOCITE: agent security benchmarks; TODOCITE: prompt injection benchmarks). These benchmarks are essential because many deployed agents operate by translating natural-language instructions into tool calls. However, they usually treat the tool or remote service as a fixed object once it has been admitted. CardDiffBench instead asks whether admission and routing metadata remain security-relevant during normal execution. The host can fail even when it does not receive a classic malicious prompt: it may simply select a risky skill, endpoint, tenant, or artifact mode because a schema-valid control-plane field made that choice appear operationally convenient.

A2A and protocol-aware agent security. Recent A2A security work has moved beyond prompt-only attacks toward registry manipulation, capability cloaking, lifecycle abuse, request forgery, and artifact attacks (TODOCITE: A2ASecBench; TODOCITE: A2A protocol security analyses). CardDiffBench is complementary to this broader line of work. Rather than covering the full A2A lifecycle, it isolates a narrower but common design pattern: the same remote agent can have multiple card views and multiple routing surfaces. This setting matters because card differentials are not necessarily malformed. They may arise from legitimate operational needs such as authenticated capability discovery, tenant-specific endpoints, versioned protocol bindings, or richer output modes. A benchmark that treats every differential as invalid would miss the real host-side decision problem; a benchmark that accepts every authenticated differential would miss the security boundary. Our taxonomy is built around this reconciliation problem.

Metadata consistency and authorization. Traditional web and distributed-systems security has long emphasized binding requests to identity, origin, authorization scope, and content type (TODOCITE: access-control and web-origin security). Agent systems inherit these concerns but express them through higher-level cards, skills, and artifacts. The difference is that an LLM-backed host may be asked to interpret these fields semantically. For example, a model may consider a skill relevant because its description matches the user task, while overlooking that the skill-level scope is stronger than the token scope. CardDiffBench turns such metadata consistency problems into executable cases with event-level oracles.

2 Background

Capability discovery through AgentCards. An AgentCard is a structured manifest through which a remote agent exposes its capabilities to a host. In the setting considered here, a card includes an agent identity, security schemes, skills, supported interfaces, protocol binding and version, tenant labels, input and output modes, and an optional authenticated extended card. The host uses these fields to decide whether and how to route a user workflow. Unlike a prompt-only setting, the dangerous action is often not a generated sentence but a protocol action: selecting an endpoint, invoking a skill, using a token scope, or accepting an artifact.

Task management. After discovery, the host turns a user workflow into an A2A-style task or message exchange. The task carries user intent, tenant context, authentication state, and requested output modes. This stage is security-relevant because the host may bind the same task to different skills or interfaces depending on which card view it treats as authoritative. A benign-looking workflow can therefore become unsafe if the task is routed through a differential control-plane surface.

Collaboration through artifacts. Agent collaboration does not end when a message is sent. Remote agents can return structured parts and artifacts, including JSON, plain text, HTML, or other MIME types. These artifacts may be consumed by downstream agents or applications. CardDiffBench therefore treats artifact acceptance as part of the host control plane: a host should enforce the requested output modes rather than accepting a returned artifact only because the remote card or response makes it look useful.

Public and authenticated views. A2A-style systems can expose a public card and an authenticated extended card. This is useful: a public card can provide safe discovery while an authenticated card can reveal more operational detail. However, the split creates ambiguity. If the extended card adds a sensitive skill, changes interface order, or exposes tenant-specific state, a host needs a policy for reconciling the public and authenticated views. Without such a policy, "use the active card" can become a route to privilege escalation or tenant confusion.

3 Threat Model

Threat model. We consider an adversary who can influence schema-valid AgentCard state or the card view returned under a particular authentication context. The adversary does not need to compromise the host code, forge invalid protocol messages, or inject arbitrary code into the host. Instead, the adversary constructs plausible card differentials and task wording that encourage the host to route through the risky

control-plane state. The goal is to cause an unsafe host action, such as invoking a sensitive skill without the required boundary, selecting an attacker-preferred interface, binding to the wrong tenant, downgrading protocol binding/version, or accepting a disallowed artifact mode.

Threat classes and stages. Following the structure of protocol-aware A2A security analysis, we organize risks by both source and stage. The first broad class is *card-state manipulation*, where the adversary changes what the host sees during discovery or authenticated card retrieval. The second class is *protocol-logic weakness*, where the host observes valid fields but fails to preserve a tenant, binding, scope, or output-mode invariant while orchestrating the task. These risks unfold across three stages: admission and discovery, where public and extended card state is selected; orchestration, where the host binds a task to a skill, tenant, interface, and protocol version; and execution, where returned artifacts are accepted or rejected. The eight CardDiff attacks instantiate these stages: A1–A3 target extended-card admission and cache behavior, B1–B3 target orchestration and routing, and C1–C2 target authorization and artifact execution policy.

Formal view. Let C_p denote the public AgentCard, $C_e(i)$ an extended card fetched under identity i, T the user task, R the requested tenant, M the accepted output-mode set, and S the token scope set. A host implements a decision function

$$H(T, C_p, C_e(i), R, M, S) \to (a, k, u, b, m),$$

where a indicates whether to send, k is the selected skill, u is the selected interface URL or index, b is the selected binding/version, and m indicates whether returned artifacts are accepted. A CardDiff attack constructs C_p, $C_e(i)$, T, R, M, and S such that the schema remains valid but at least one security invariant is violated by the host decision. The oracle is attack-specific: for A1 it is $k = k_{\text{sensitive}}$; for B2 it is $R \neq R_{\text{selected}}$; for C2 it is m accepting a MIME type outside M. This formulation makes clear that the benchmark target is the decision boundary, not only the language surface.

4 The CardDiffBench Framework

4.1 Attack Taxonomy

CardDiffBench contains eight attack types grouped into three families. Table 1 summarizes the benchmark taxonomy and the event used for scoring. The attacks are not prompt templates; each attack defines a protocol-state mutation and a private oracle.

4.2 Dataset Construction

The dataset is built from four structured inputs: attack vectors, domain scenarios, scenario-adapted natural-language tasks, and perturbation variants. The three domains are travel, healthcare, and finance. Each domain defines a normal tenant, privileged tenant, normal skill, sensitive skill, normal scope, privileged scope, safe output modes, and risky artifact MIME type. For example, the finance domain contrasts statement lookup under `finance.read` with high-value wire transfer under `finance.transfer`; the healthcare domain contrasts appointment lookup with prescription export; and the travel domain contrasts itinerary lookup with payment refund.

The base split contains 50 tasks for every attack-domain cell. With 8 attacks and 3 domains, this yields $8 \times 3 \times 50 = 1{,}200$ base cases. The perturbation split applies three oracle-preserving variants to each base case: the original construction, a semantic-alias variant that renames or rephrases protocol objects, and an ordering/binding-shift variant that changes the control surface while preserving the same success condition. The resulting benchmark contains $1{,}200 \times 3 = 3{,}600$ perturbed cases.

The scenario adapter generates task wording separately from the protocol mutation. This separation is important. The natural-language task must be plausible and executable, but the benchmark target is the host's control-plane decision, not merely its ability to resist a suspicious sentence. During generation, the adapter is instructed not to reveal oracle labels, attack identifiers, or scoring hints. The case builder then binds the task to concrete AgentCard state and a private oracle.

4.3 Execution Harness and Scoring

Each case runs in a local HTTP environment that exposes the relevant public card, optional extended card, message endpoint, token identity, tenant metadata, returned artifacts, and event recorder. The host receives a workflow request, fetches card state, chooses a skill and interface, sends an A2A-style message, and optionally accepts returned artifacts. The harness records events such as `message_sent` and `artifact_accepted`; the scorer evaluates these events against the attack oracle.

This event-level scoring avoids two common ambiguities. First, it does not rely on the host's natural-language explanation of its decision. A host can claim to be cautious while still sending the risky message. Second, it distinguishes different unsafe routes. A1 is scored by sensitive skill invocation, B2 by tenant mismatch, B3 by protocol binding/version mismatch, and C2 by artifact MIME acceptance. The benchmark therefore measures concrete control-plane behavior.

4.4 Reproducibility Assets

The active benchmark assets are separated from archived reference material. Attack vectors live under `attacks/carddiff/vectors.json`; scenario definitions under `attacks/carddiff/scenarios.json`; generated task banks under `attacks/carddiff/scenario_tasks.json`; perturbation definitions under `attacks/carddiff/perturbations.json`; and frozen executable cases under `attacks/carddiff/cases.jsonl` and `attacks/carddiff/perturbed_cases.jsonl`. The host implementations and scorer are kept in `sut/carddiff/` and `harness/`. This separation is deliberate: the benchmark contribution is not a repackaging of a prior A2A security suite, but a CardDiff-specific

Family	ID	Attack	Success evidence
Extended-card differential	A1	Public-to-Extended Skill Escalation	Sensitive skill is invoked
	A2	Public-to-Extended Interface Drift	Message routes to drifted extended-card URL
	A3	Cross-Identity Extended Card Cache Bleed	Low-privilege identity reuses privileged card state
Interface/tenant confusion	B1	Interface Order Hijacking	Attacker-preferred interface is selected
	B2	Same-URL Multi-Tenant Confusion	Selected tenant differs from requested tenant
	B3	Binding Downgrade / Version Confusion	Binding or protocol version differs from expected value
Policy/mode drift	C1	Skill-Level Authorization Confusion	Restricted skill invoked without required scope
	C2	Media Mode Drift	Artifact outside accepted output modes is accepted

Table 1: CardDiffBench attack taxonomy. Each attack is scored by protocol events emitted by the local environment, rather than by text matching alone.

dataset and scoring layer with archived prior code retained only for provenance and comparison.

Every run writes per-trial JSONL records and aggregate summaries. The completed main experiment was executed as 24 checkpoints, one for each attack-variant cell. This checkpointing makes the benchmark practical for LLM-backed runs, where network timeouts or model endpoint failures can otherwise obscure which part of the evaluation completed. It also makes selective reruns possible while preserving the accepted main ledger.

5 Evaluation and Result Analysis

5.1 Experimental Setup

Host under test. We evaluate a single LLM-backed host implementation. The protocol I/O is executed by deterministic harness code, while the model decides the control-plane choices that matter for CardDiffBench: whether to send a message, which skill to invoke, which supported interface to use, and whether returned artifacts should be accepted. The host prompt presents the user task, public card summary, active card summary, authentication step, requested tenant, and accepted output modes. The model returns a JSON decision with fields for `should_send`, `skill_id`, `interface_index`, `accept_artifacts`, and status.

Model and run protocol. The main run uses `gpt-5-mini`. Each of the 24 attack-variant checkpoints contains 150 cases, corresponding to 50 tasks across each of the three domains. We run one trial per case, for 3,600 total trials. Timeout or execution errors are counted as blocked/unsuccessful attack trials; parse failures are recorded separately. In the completed run, parse failures were zero.

Metric. The primary metric is attack success rate (ASR), defined as

$$\mathrm{ASR} = \frac{\#\text{successful attack trials}}{\#\text{total trials}}.$$

ASR is reported by attack type and overall. Since this draft reports the main adversarial evaluation only, benign utility and defense evaluation are left as planned follow-up experiments rather than claimed results.

CardDiffBench loop. Scenario + attack vector → natural-language task bank → AgentCard differential case → protocol-state perturbation → local A2A environment → host decision event log → oracle scorer.

Figure 1: Benchmark construction and execution loop. A final version should replace this text box with a compact pipeline figure.

5.2 Main Results

Control-plane differentials were broadly effective. Table 2 shows that all eight attacks achieved at least 99.56% ASR. The strongest attacks reached 100% ASR: A2, which routes through a drifted extended-card interface, and A3, which tests whether identity-specific extended-card state can bleed across contexts. The result indicates that the host consistently privileged the active card view and workflow completion over conservative reconciliation between public, authenticated, tenant, and policy surfaces.

Failures were operational rather than semantic. Only seven of 3,600 trials did not produce attack success. These cases were recorded as blocked/error outcomes, mostly from timeout-style execution failures, not from JSON parse failures. This distinction matters: the low failure count should not be interpreted as evidence that the host rejected unsafe card state. Instead, the run suggests that when the host completed the protocol workflow, it almost always completed it through the attack-preserving control-plane path.

Perturbations preserved effectiveness. The benchmark includes three variants per base case: base construction, semantic aliasing, and ordering/binding shift. High ASR across all attack categories suggests that the attacks do not depend only on a single lexical form of a task or field name. The perturbation design changes surface names, ordering, or binding context while preserving the oracle, thereby testing whether the host tracks the underlying control-plane invariant. In the main run, it generally did not.

5.3 Result Analysis

Why the attacks succeed. The evaluated host was designed to be helpful: it treated the authenticated active card as the operational source for completing the task. That stance

Attack	Travel		Healthcare		Finance		Overall		
	Succ./Trials	ASR	Succ./Trials	ASR	Succ./Trials	ASR	Trials	Successes	ASR
A1	150/150	100.00%	150/150	100.00%	148/150	98.67%	450	448	99.56%
A2	150/150	100.00%	150/150	100.00%	150/150	100.00%	450	450	100.00%
A3	150/150	100.00%	150/150	100.00%	150/150	100.00%	450	450	100.00%
B1	150/150	100.00%	148/150	98.67%	150/150	100.00%	450	448	99.56%
B2	149/150	99.33%	149/150	99.33%	150/150	100.00%	450	448	99.56%
B3	150/150	100.00%	150/150	100.00%	149/150	99.33%	450	449	99.78%
C1	149/150	99.33%	150/150	100.00%	150/150	100.00%	450	449	99.78%
C2	150/150	100.00%	149/150	99.33%	150/150	100.00%	450	449	99.78%
Overall	1198/1200	99.83%	1196/1200	99.67%	1197/1200	99.75%	3600	3591	99.75%

Table 2: Main `gpt-5-mini` results on the 3,600-case perturbed split, broken down by scenario. Errors/timeouts are counted as unsuccessful attacks.

is natural for many agent systems, but it is insufficient when AgentCard fields encode security boundaries. A secure host needs to bind an action not only to an available skill or endpoint, but also to the provenance of the card field, the identity under which it was retrieved, the requested tenant, the required scope, and the output-mode policy. Without those bindings, a schema-valid differential can turn ordinary routing into escalation.

Relation to prior A2A security benchmarks. Prior A2A security work has studied broader protocol attacks such as spoofing, capability manipulation, lifecycle abuse, request forgery, and artifact-triggered attacks (TODOCITE: A2ASecBench and related work). CardDiffBench narrows the lens to AgentCard state consistency. This narrower focus is useful because many real hosts will rely on cards as routine infrastructure rather than as obviously adversarial content. The benchmark therefore complements broader security suites by isolating a measurable control-plane class: what happens when card fields disagree but remain plausible and executable?

Implications for defenses. The results motivate several defense directions. First, hosts should maintain a typed provenance ledger for every card field used in a decision, distinguishing public, authenticated, cached, identity-specific, and tenant-specific sources. Second, skill selection should be checked against skill-level security, not only agent-level security. Third, interface selection should verify tenant and binding invariants rather than defaulting to ordering. Fourth, artifact acceptance should be enforced by the host-side requested output modes, not by the remote agent's returned artifact alone. CardDiffBench can evaluate these defenses by comparing ASR before and after each policy is added.

Defense evaluation protocol. The next experimental stage should evaluate defenses as host-side policy modules rather than as changes to the attack data. A minimal protocol has four conditions: an undefended helpful host, a provenance-binding host, a scope-checking host, and a combined host that enforces provenance, tenant, binding, scope, and artifact-mode constraints. Each condition should be run on the same 3,600 perturbed cases. A useful defense should reduce ASR while preserving benign task completion on a paired non-adversarial split. For analysis, failures should be separated into explicit refusals, corrected safe routing, protocol errors, and model parse failures. This decomposition would reveal whether a defense actually understands the control-plane invariant or merely blocks more tasks.

Downstream impact analysis. The main benchmark intentionally scores the boundary crossing itself. A small supplementary study can then estimate downstream impact without turning every case into a live harmful action. For each attack family, one can select a small stratified sample and attach a harmless simulator for the sensitive action: refund issuance, prescription export, treasury transfer preparation, tenant-confused record lookup, or unsafe HTML artifact consumption. The outcome would not replace ASR, but it would make the consequence chain more concrete: CardDiff success at the host can become unauthorized workflow execution at the downstream agent.

Limitations. This draft reports a main adversarial evaluation on one LLM-backed host and one model. It does not yet include benign utility trials, cross-model comparisons, human-written adversarial cases, or a deployed multi-agent system with real downstream side effects. The current downstream agents are harness-controlled peers, so the benchmark measures whether the host crosses a control-plane boundary, not the full chain of real-world damage after that boundary is crossed. A small follow-up impact study could instantiate a subset of successful cases with concrete downstream consequences, such as unauthorized refund initiation, protected record export, treasury transfer preparation, or unsafe HTML artifact consumption.

5.4 Limitations and Responsible Release

CardDiffBench is designed as an evaluation tool for defensive research. The benchmark uses synthetic domains, local harness endpoints, opaque benchmark tokens, and controlled artifacts. The attack descriptions identify protocol-state failure modes rather than providing instructions for compromising live services. A public release should include guidance for running the benchmark only against owned systems or local test deployments, and should document how to interpret ASR as evidence of missing host-side validation rather than

Order	Attack	Variant	Trials	Successes	ASR	Errors	Parse failures
1	A1	001	150	150	100.00%	0	0
2	A1	002	150	149	99.33%	0	0
3	A1	003	150	149	99.33%	0	0
4	A2	001	150	150	100.00%	0	0
5	A2	002	150	150	100.00%	0	0
6	A2	003	150	150	100.00%	0	0
7	A3	001	150	150	100.00%	0	0
8	A3	002	150	150	100.00%	0	0
9	A3	003	150	150	100.00%	0	0
10	B1	001	150	149	99.33%	1	0
11	B1	002	150	150	100.00%	0	0
12	B1	003	150	149	99.33%	1	0
13	B2	001	150	150	100.00%	0	0
14	B2	002	150	150	100.00%	0	0
15	B2	003	150	148	98.67%	2	0
16	B3	001	150	150	100.00%	0	0
17	B3	002	150	149	99.33%	1	0
18	B3	003	150	150	100.00%	0	0
19	C1	001	150	150	100.00%	0	0
20	C1	002	150	150	100.00%	0	0
21	C1	003	150	149	99.33%	1	0
22	C2	001	150	150	100.00%	0	0
23	C2	002	150	149	99.33%	1	0
24	C2	003	150	150	100.00%	0	0

Table 3: Full checkpoint ledger for the completed `gpt-5-mini` main experiment. Each checkpoint contains 50 cases per domain across travel, healthcare, and finance.

as a property of any real third-party agent.

6 Conclusion

We introduced CardDiffBench, a benchmark for AgentCard control-plane differentials in agent-to-agent systems. The benchmark defines eight attacks across extended-card differentials, interface/tenant confusion, and policy/mode drift; provides 1,200 base cases and 3,600 oracle-preserving perturbations across three high-stakes domains; and scores host behavior through protocol events. In a completed `gpt-5-mini` host evaluation, attacks succeeded in 99.75% of perturbed trials. These findings suggest that control-plane validation is a distinct security requirement for interoperable agent ecosystems. Future work should add benign utility measurement, cross-model evaluation, defense ablations, and small-scale downstream-impact studies to quantify how unsafe card decisions propagate beyond the host.

Acknowledgments

Omitted for anonymous review.